Zadanie z kontenera DI, AutoFac (lab 3) ver 1.2:

Cel: Prakczne zastosowanie wiedzy na temat kontenera DI Autofac, włączając w to różne sposoby wstrzykiwania zależności, konfigurację imperatywną i deklaratywną oraz zaawansowane zarządzanie czasem życia komponentów za pomocą zasięgów (Lifetime Scopes).

Część 1: Implementacja komponentów (24%)

1. **(4%)** Zaimplementuj klasę `Worker`, posiadającą metodę `Work(string a, string b)`.
   - Klasa `Worker` powinna korzystać z interfejsu `ICalculator`.
   - Interfejs ten powinien udostępniać metodę `string Eval(string a, string b)`.
   - Metoda `Work` powinna wypisywać na konsolę wynik wywołania `Eval(a, b)`.
   - Obiekt `ICalculator` powinien być wstrzykiwany przez **konstruktor**.
2. **(4%)** Zaimplementuj klasę `CatCalc` implementującą `ICalculator`. Wynikiem metody `Eval` powinna być konkatenacja (złączenie) napisów `a` i `b`.
3. **(4%)** Zaimplementuj klasę `PlusCalc` implementującą `ICalculator`. Wynikiem metody `Eval` powinna być suma liczb `a` i `b` (np. `Eval("123", "111")` powinno zwrócić `"234"`).
4. **(4%)** Zaimplementuj klasę `StateCalc` implementującą `ICalculator`. Wynikiem metody `Eval` powinna być konkatenacja napisów `a`, `b` i kolejnej liczby, np. `Eval("a", "b")` -> `"ab1"`, kolejne `Eval("c", "d")` -> `"cd2"`. Numer początkowy powinien być przekazywany w konstruktorze.
5. **(4%)** Zaimplementuj klasę `Worker2`, która różni się od `Worker`a tylko sposobem wstrzykiwania `ICalculator`a (przez **metodę** `SetCalculator(ICalculator calc)`).
6. **(4%)** Zaimplementuj klasę `Worker3`, która różni się od `Worker`a tylko sposobem wstrzykiwania `ICalculator`a (przez publiczną **właściwość** `Calculator`).

Część 2: Konfiguracja kontenera Autofac (21%)

Skonfiguruj kontener Autofac tak, aby spełniał poniższe wymagania. Każdą konfigurację należy zaimplementować zarówno **imperatywnie (w kodzie C#)**, jak i **deklaratywnie (w pliku `appsettings.json`)**. Autofac przestał wspierać bezpośrednio atrybut `name` do deklaratywnego tworzenia nazwanych komponentów, więc ostatnie trzy wymagania należy zaimplementować używając metadanych w pliku JSON (przykład w instrukcji):

- **(3%)** Domyślne obiekty `Worker` korzystały z `CatCalc`.
- **(3%)** Domyślne obiekty `Worker2` korzystały z `PlusCalc`.
- **(3%)** Domyślne obiekty `Worker3` korzystały z `CatCalc`.
- **(3%)** `StateCalc` był zarejestrowany jako **singleton** (`SingleInstance`).
- **(3%)** Nazwane obiekty `Worker` (z nazwą `"state"`) korzystały ze `StateCalc`.
- **(3%)** Nazwane obiekty `Worker2` (z nazwą `"state"`) korzystały ze `StateCalc`.
- **(3%)** Nazwane obiekty `Worker3` (z nazwą `"state"`) korzystały ze `StateCalc`.

Część 3: Zaawansowane zarządzanie czasem życia (25%)

1. **(5%) Implementacja:** Stwórz interfejs `IUnitOfWork` oraz jego implementację `UnitOfWork`. Klasa `UnitOfWork` powinna w konstruktorze generować i przechowywać publicznie dostępny, unikalny identyfikator `Guid`.
2. **(5%) Konfiguracja:** Zarejestruj `IUnitOfWork` w kontenerze z czasem życia `InstancePerLifetimeScope`.
3. **(5%) Implementacja:** Zaimplementuj interfejs `ITransactionContext` oraz klasy `TransactionContext`, `StepOneService`, `StepTwoService` i `TransactionProcessor` zgodnie z przykładem z instrukcji laboratoryjnej.
4. **(10%) Konfiguracja:** Zarejestruj `ITransactionContext` w kontenerze z czasem życia `InstancePerMatchingLifetimeScope("transaction")`, a pozostałe serwisy (`StepOneService` itd.) w standardowy sposób.

Część 4: Testy jednostkowe (30%)

- **(20%)** Dla **każdego scenariusza konfiguracji** z **Części 2** napisz test jednostkowy, który weryfikuje poprawność działania (np. czy `Worker` po rozwiązaniu z kontenera zwraca wynik konkatenacji).
  - Każdy test musi być wykonany dwukrotnie – raz dla konfiguracji imperatywnej i raz dla deklaratywnej (każda za 10%). Można w tym celu wykorzystać atrybuty `[DataRow(true)]` / `[DataRow(false)]` (MSTest) lub `[Theory]` / `[InlineData(...)]` (xUnit).
  - Napisz osobny test weryfikujący, czy `StateCalc` jest rzeczywiście singletonem (czy kolejne rozwiązania z kontenera zwracają ten sam obiekt).
- **(5%)** Napisz testy jednostkowe dla **Części 3**, które weryfikują:
  - Czy `IUnitOfWork` (`InstancePerLifetimeScope`) zwraca **tę samą instancję** w ramach jednego zasięgu, ale **różne instancje** w różnych zasięgach.
- **(5%)** Napisz testy jednostkowe dla **Części 3**, które weryfikują:
  - Czy `ITransactionContext` (`InstancePerMatchingLifetimeScope`) zwraca **tę samą instancję** dla `StepOneService` i `StepTwoService` w ramach jednego, otagowanego zasięgu (`BeginLifetimeScope("transaction")`).

Framework testowy: Można wykorzystać dowolny framework, np. MSTest, NUnit, xUnit.